Spark Optimization

Applications Using Scala

Kodjo Klouvi

kodjo@osekoo.com

COURSE PLAN

Learning outcomes

- Explain jobs, stages, tasks, partitions, and shuffle boundaries.
- Read a formatted physical plan and connect it to Spark UI metrics.
- Reduce scans, use efficient expressions, and keep large results off the driver.
- Choose partitions, join strategies, and persistence from evidence.
- Diagnose skew, small files, spill, and repeated computation.

METHOD Measure a baseline, change one factor, verify correctness, and compare the same metrics.

CONTINUOUS CASE STUDY

Course dataset

Path	Purpose	Deliberate characteristic
countries	Tiny dimension	Broadcast candidate
customers	Customer dimension	Segments and country
orders_partitioned	Main fact table	Partitioned by order_date
orders_small_files	Alternative order input	Fragmented files
events	Second fact table	Two actions for cache reuse

CLUE customer_id 42 receives about 22% of orders. The wide debug_payload column is unnecessary for revenue analysis.

SETUP

Dataset generation

```
$env:COURSE_DATA_PATH = "C:\\tmp\\spark-course-data"
$env:CUSTOMER_COUNT = "20000"
$env:ORDER_COUNT = "200000"
$env:EVENT_COUNT = "300000"
$env:NORMAL_PARTITIONS = "8"
$env:SMALL_FILE_PARTITIONS = "40"

spark-shell -i generate_course_data.scala
```

- The small profile validates every lab on a classroom laptop.
- Use the standard or cluster profile for visible distributed costs.
- Restart the Spark shell before recording baseline measurements.

SETUP

Dataset loading

```
import org.apache.spark.sql.functions.*
import org.apache.spark.storage.StorageLevel

val root = sys.env.getOrElse("COURSE_DATA_PATH", "./spark-course-data")
val orders = spark.read.parquet(s"$root/orders_partitioned")
val fragmentedOrders = spark.read.parquet(s"$root/orders_small_files")
val customers = spark.read.parquet(s"$root/customers")
val countries = spark.read.parquet(s"$root/countries")
val events = spark.read.parquet(s"$root/events")
```

BASELINE Record runtime, row count, scan bytes, shuffle read and write, spill, task count, and the slowest task.

FOUNDATIONS

Spark execution architecture

Component	Responsibility	Optimization concern
Driver	Builds plans and schedules work	collect, metadata, large closures
Executor	Runs tasks and stores partitions	CPU, memory, GC, spill
Partition	Unit of distributed data	Balance and task size
Shuffle	Redistributes data by key	Network, disk, sort, serialization

RULE The driver coordinates work. Keep large datasets distributed across executors.

EXECUTION MODEL

Transformations and actions

Transformations	Actions
Build a lazy plan	Trigger execution
filter, select, join, groupBy	count, collect, show, write
Spark can optimize the expression tree	An action can recompute an uncached lineage
Usually no job at definition time	Jobs, stages, and tasks run

```
val paid = orders.filter($"status" === "PAID")  // lazy
val rows = paid.count()                         // action
```

EXECUTION MODEL

Jobs, stages, tasks, and partitions

Concept	Meaning	Performance signal
Job	Work triggered by an action	Repeated actions may repeat lineage
Stage	Tasks between shuffle boundaries	Exchange often starts a new stage
Task	Work on one partition	Slowest task can delay the stage
Partition	Slice of distributed data	Too few or too many both cost
Shuffle	Cross-executor redistribution	Read, write, spill, fetch wait

QUERY PLANS

Logical and physical plans

```
val q = orders
  .filter($"status" === "PAID")
  .join(customers, Seq("customer_id"))
  .groupBy($"segment").agg(sum($"amount"))

q.explain("formatted")
q.explain("cost")
```

- Scan: columns, partition filters, and pushed filters.
- Exchange: shuffle boundaries and partitioning.
- Join: broadcast hash, sort merge, shuffled hash, or nested loop.
- Adaptive plan: initial and final operators may differ.

CORRECTION explain(extended = true) shows parsed, analyzed, optimized, and physical plans. The formatted mode supports operator-level reading.

EXERCISE

Plan detective

EXERCISE 1 | 5 minutes

Inspect the January paid-order query before running it.

```
val januaryPaid = orders
  .filter($"order_date".between("2026-01-01", "2026-01-31"))
  .filter($"status" === "PAID")
  .select("customer_id", "amount", "country")

januaryPaid.explain("formatted")
```

- Find the scan and ReadSchema.
- Find PartitionFilters and pushed filters.
- Predict the number of input partitions and scan files.

SOLUTION

Plan detective solution

SOLUTION 1

- ReadSchema should contain only the required columns and partition metadata.
- The order_date condition should appear as a partition filter.
- The status condition may appear as a pushed filter depending on source capabilities.
- Confirm the final plan after execution because AQE can change operators.

DATA REDUCTION

Read less data

- Select only columns required by the result.
- Filter partition columns early so Spark can skip directories.
- Prefer Parquet or ORC for analytical scans.
- Avoid high-cardinality storage partition columns.
- Compact tiny files when scheduling and file-open overhead dominate.

DATASET CLUE Selecting revenue columns removes debug_payload from ReadSchema. Filtering order_date demonstrates partition pruning.

EFFICIENT APIS

Built-in expressions and UDFs

```
// Optimizer-visible expression
val clean = customers.withColumn(
  "email_domain", lower(element_at(split($"email", "@"), 2))
)

// Use a UDF only when built-in expressions cannot express the rule
val domainUdf = udf((email: String) => email.split("@").last.toLowerCase)
```

- Built-in expressions remain visible to Catalyst.
- UDFs can restrict optimization and add call or serialization overhead.
- Measure typed Dataset and object-conversion costs when type safety matters.

EXERCISE

Projection and filtering

EXERCISE 2 | 6 minutes

Rewrite the pipeline so Spark carries only required data into the join.

```
val result = orders
  .join(customers, Seq("customer_id"))
  .filter($"status" === "PAID")
  .groupBy($"segment")
  .agg(sum($"amount").as("revenue"))
```

- Identify required columns on both sides.
- Move safe filters before the join.
- State which plan and UI metrics should improve.

SOLUTION

Projection and filtering solution

SOLUTION 2

```
val paid = orders
  .filter($"status" === "PAID")
  .select("customer_id", "amount")

val segments = customers.select("customer_id", "segment")

val result = paid.join(segments, Seq("customer_id"))
  .groupBy($"segment")
  .agg(sum($"amount").as("revenue"))
```

VERIFY Compare scan bytes, shuffle volume, join input rows, runtime, and revenue totals. The output must remain identical.

DISTRIBUTED WORK

Partitions and shuffle

Operation	Behavior	Typical use
repartition(n)	Full redistribution	Rebalance before wide work
repartition(n, key)	Hash distribution by key	Align work around a join or aggregate key
coalesce(n)	Usually reduces without full shuffle	Reduce tiny partitions after filtering
partitionBy on write	Creates a storage layout	Enable downstream pruning

NO MAGIC NUMBER Choose partition counts from data size, cluster cores, task duration, skew, and target file size.

DISTRIBUTED WORK

Join strategies

Situation	Likely strategy	Evidence to check
Tiny dimension and large fact	Broadcast hash join	Broadcast size and executor memory
Two large equality inputs	Sort merge or shuffled hash	Shuffle volume and balance
Very frequent keys	AQE skew handling or selective salting	Slowest task versus median
Non-equality or cross condition	Possible nested-loop behavior	Cardinality explosion

```
val activeCountries = countries.filter($"active" === true)
val enriched = orders.join(broadcast(activeCountries),
  $"country" === $"country_code")
```

EXERCISE

Broadcast comparison

EXERCISE 3 | 8 minutes

Compare the normal join with an explicit broadcast of countries.

```
val normalJoin = orders.join(countries, $"country" === $"country_code")
val broadcastJoin = orders.join(broadcast(countries),
  $"country" === $"country_code")

normalJoin.explain("formatted")
broadcastJoin.explain("formatted")
```

- Record the physical join operator.
- Compare shuffle read and write.
- Explain why broadcasting customers may be unsafe at larger scale.

SOLUTION

Broadcast comparison solution

SOLUTION 3

- countries should produce BroadcastExchange and BroadcastHashJoin when the hint is accepted.
- The large orders side should avoid a join shuffle.
- A broadcast copy lives on every executor, so memory limits the safe size.
- Prefer credible statistics and Spark's automatic choice when possible.

DISTRIBUTED WORK

Data skew

```
orders.groupBy($"customer_id").count()
  .orderBy(desc("count"))
  .show(10, truncate = false)
```

- A skewed key creates an oversized partition and a long-running task.
- Compare maximum task duration and input with the median.
- AQE can split skewed shuffle partitions at runtime.
- Process exceptional keys separately or salt only confirmed hot keys.

KNOWN HOT KEY customer_id 42 appears in about 22% of generated orders.

EXERCISE

Skew investigation

EXERCISE 4 | 6 minutes

Prove that the workload is skewed before proposing a fix.

- Run the customer frequency query and identify the top key.
- Run a customer aggregation and open stage details.
- Record median and maximum task duration, shuffle read, and spill.
- Check the final adaptive plan for skew handling.
- Propose the least complex fix supported by evidence.

SOLUTION

Skew investigation solution

SOLUTION 4

- Key 42 should dominate the frequency distribution.
- One or a few reduce tasks may process much more input than the median.
- Keep AQE enabled and inspect whether Spark splits the skewed partition.
- If AQE is insufficient, isolate key 42 when business logic allows.
- Adding executors alone does not split one oversized partition.

CACHE AND STORAGE

Persistence decisions

Cache when	Avoid caching when
The same expensive result feeds several actions	The data is used once
Recomputation is expensive	The source scan is cheap
The persisted form fits the storage budget	It causes eviction, GC, or spill
A stable snapshot is acceptable	Every action must read new source data

```
val base = events.filter($"event_date" >= "2026-10-15")
  .persist(StorageLevel.MEMORY_AND_DISK)
// two actions use base
base.unpersist()
```

EXERCISE

Cache reuse

EXERCISE 5 | 6 minutes

Measure two actions on the same filtered event data with and without persistence.

```
val base = events.filter($"event_type" =!= "view")

base.groupBy("event_type").count().count()
base.groupBy("user_id").agg(sum("value")).count()
```

- Record both action times without caching.
- Persist base and repeat in a fresh run.
- Separate cache population time from reuse time.
- Inspect the Storage tab and unpersist afterward.

SOLUTION

Cache reuse solution

SOLUTION 5

```
val base = events.filter($"event_type" =!= "view")
  .persist(StorageLevel.MEMORY_AND_DISK)

base.groupBy("event_type").count().count() // populates cache
base.groupBy("user_id").agg(sum("value")).count() // reuses cache
base.unpersist()
```

INTERPRETATION Compare total time for both actions. Caching can lose when reuse does not repay population and storage costs.

STORAGE

Small files and output layout

Symptom	Cost	Response
Many tiny input files	Listing, open, and scheduling overhead	Compact to a sensible file size
repartition(1)	One bottleneck task	Write distributed output
High-cardinality partitionBy	Many directories and tiny files	Choose a lower-cardinality filter column
Unpartitioned date scans	Reads unrelated dates	Partition by a commonly filtered date

```
fragmentedOrders.coalesce(32)
  .write.mode("overwrite").parquet(s"$root/orders_compacted")
```

OBSERVABILITY

Spark UI diagnostic workflow

Area	Look for	Question
SQL	Physical plan and runtime statistics	Which operator dominates?
Jobs	Repeated actions	Is upstream work recomputed?
Stages	Shuffle, spill, task distribution	Is the stage skewed?
Executors	GC, input, failures	Is pressure isolated or global?
Storage	Cached size and fraction	Does persistence pay?
Environment	Effective properties	Did the intended setting apply?

HABIT Compare the slowest task with the median. Averages hide skew and stragglers.

SCALA

Scala-specific Spark tips

- Prefer DataFrame and SQL expressions for relational processing.
- Use typed Datasets when type safety justifies conversion costs.
- For RDD aggregation, prefer reduceByKey or aggregateByKey over groupByKey.
- Use mapPartitions to amortize expensive client setup.
- Keep closures small and broadcast genuinely small lookup data.
- Design external side effects for task retries and speculative execution.

CORRECTION Scala can avoid Python boundary costs in some workloads, but language choice alone does not guarantee a faster Spark job.

EXERCISE

End-to-end optimization challenge

EXERCISE 6 | 10 minutes

Produce paid revenue by customer segment from the generated dataset.

- Create a baseline and validate row counts and revenue totals.
- Reduce columns and rows before the join.
- Inspect the join strategy, shuffle volume, and task distribution.
- Address skew only if metrics show a critical straggler.
- Write distributed Parquet output and report before-and-after evidence.

DELIVERABLE Provide the final Scala pipeline, formatted plan, five key metrics, and a short explanation of each change.

SOLUTION

End-to-end challenge solution

SOLUTION 6

```
val paid = orders.filter($"status" === "PAID")
  .select("customer_id", "amount")
val segments = customers.select("customer_id", "segment")

val revenue = paid.join(segments, Seq("customer_id"))
  .groupBy("segment").agg(sum("amount").as("revenue"))

revenue.explain("formatted")
revenue.write.mode("overwrite").parquet(s"$root/revenue_by_segment")
```

CONDITION Do not broadcast customers without verifying its size. Filter and projection may be the correct first optimization.